Datathon 2026: Challenge Guide

Introduction

Welcome to the Engineering Datathon! This event challenges your ability to handle real-world data, build predictive models, and design scalable systems. You are expected to deliver functional prototypes or high-fidelity simulations that address the following problem statements.

Track 1: Smart Traffic Management System

Problem Statement:
Urban areas suffer from high congestion because traffic signals operate on static timers regardless of actual vehicle density. Participants must develop an **AI-driven Adaptive Signal Control System** that analyzes live traffic feeds to dynamically adjust "Green Light" durations. The system should prioritize emergency vehicles (ambulances/fire trucks) and reduce "dead-time" at empty intersections.

- **Tech Stack:**
  - **Computer Vision:** YOLO (v8/v10), OpenCV.
  - **Programming:** Python.
  - **Simulation:** SUMO (Simulation of Urban MObility) or Pygame for visualization.
  - **Deployment:** Flask/FastAPI for the backend dashboard.
- **Expected Output:**
  - A dashboard showing real-time vehicle counts per lane.
  - A simulation demonstrating a reduction in average waiting time compared to static timers.
  - Automated detection and "Green Wave" triggering for emergency vehicles.

Track 2: Railway Ticket Management & Predictive Analysis

Problem Statement:

Traditional railway systems struggle with "Waitlist" uncertainty and seat wastage. Participants need to create an **Intelligent Booking & Prediction Engine**. This system should not only manage bookings and cancellations but also use historical data to provide a **PNR Confirmation Probability**. Additionally, it should suggest "Alternative Route/Train" options if the primary choice is unavailable.

- **Tech Stack:**
  - **Backend:** Java (Spring Boot) or Node.js.
  - **Database:** PostgreSQL or MongoDB (to handle PNR records).
  - **Data Science:** Scikit-learn, Pandas, XGBoost (for confirmation prediction).
  - **Frontend:** React.js or Angular.
- **Expected Output:**
  - A functional booking portal with PNR generation.
  - A machine learning model with $>80\%$ accuracy in predicting waitlist confirmation.
  - A "Dynamic Seat Allocation" algorithm to maximize occupancy.

Track 3: AI Content Detector

Problem Statement:

With the rise of Large Language Models (LLMs), distinguishing between human-written and AI-generated content is critical for academic integrity and misinformation control. Participants must build a **Deep Learning Classifier** that detects text generated by models like GPT-4, Claude, or Gemini. The tool should provide a "Perplexity" score and highlight specific segments likely to be AI-generated.

- **Tech Stack:**
  - **Natural Language Processing (NLP):** Hugging Face Transformers, PyTorch/TensorFlow.
  - **Pre-trained Models:** RoBERTa, BERT, or T5.
  - **Data:** Web-scraped datasets or the "Deepfake Text" dataset.
  - **Frontend:** Streamlit or Tailwind CSS.
- **Expected Output:**
  - A web interface where users can paste text for analysis.
  - A "Probability Score" (%) indicating AI vs. Human origin.
  - A report on the model's performance (F1-score, Precision, Recall).

Track 4: AI-Powered Smart Agriculture (IoT & CV)

Problem Statement: Farmers often face crop loss due to undetected pests or nutrient deficiencies. Participants must build a **Crop Health Monitoring System** that analyzes images of leaves to identify diseases and recommend specific fertilizers or pesticides. The system should also predict the "Optimal Harvest Date" based on historical weather patterns.

- **Tech Stack:** * **DL Frameworks:** TensorFlow/Keras.
  - **Datasets:** PlantVillage dataset.
  - **Mobile/Web:** Flutter or React Native (for farmer accessibility).
- **Expected Output:**
  - A mobile app where a user can upload a photo of a leaf and get a diagnosis.
  - A localized weather integration dashboard showing soil moisture levels.

Track 5: Blockchain-Based Supply Chain for Pharmacy

Problem Statement: The pharmaceutical industry is plagued by counterfeit drugs. Participants are tasked with creating a **Transparent Drug Tracking System** using Blockchain. Every step from the manufacturer to the local pharmacy must be logged on a ledger to ensure the medicine's authenticity.

- **Tech Stack:** * **Blockchain:** Ethereum, Solidity (Smart Contracts).
    - **Web3:** Ethers.js or Web3.js.
    - **Backend:** Node.js.
- **Expected Output:**
    - A decentralized app (dApp) where a customer can scan a QR code to see the entire "Chain of Custody" of a medicine bottle.
    - Immutable logs of temperature-sensitive drug storage data.

Track 6: Personalized Mental Health Chatbot (NLP)

Problem Statement: With rising stress levels, many students don't have immediate access to counseling. Build an **Empathetic AI Companion** that uses Sentiment Analysis to gauge a user's mood. It should provide coping mechanisms for anxiety/stress and, most importantly, trigger an "Emergency Alert" to a human counselor if it detects signs of self-harm.

- **Tech Stack:** * **NLP:** OpenAI API (GPT-4) or Llama 3 (via Ollama).
    - **Sentiment Analysis:** VADER or TextBlob.
    - **Frontend:** Next.js.
- **Expected Output:**
    - A real-time chat interface with high responsiveness.
    - A "Mood Tracker" graph showing the user’s emotional trends over a week.

Track 7: Fraud Detection in UPI/Digital Payments

Problem Statement: Digital payment fraud is evolving rapidly. Participants must develop a **Real-Time Fraud Detection Engine** that flags suspicious transactions based on anomalies (e.g., unusual location, sudden high-value transfer, or rapid-fire small transactions).

- **Tech Stack:** * **ML Algorithms:** Isolation Forest, Random Forest, or Autoencoders.
    - **Data Processing:** Apache Kafka or Spark (for stream processing).
    - **Database:** Redis (for fast lookups).
- **Expected Output:**
    - A system that classifies transactions as "Safe," "Flagged," or "Blocked."
    - A report showing the "False Positive" rate (avoiding blocking legitimate users).

Track 8: Automated Resume Screener & Job Matcher

Problem Statement: HR departments receive thousands of resumes that don't match job descriptions (JD). Build a **Resume Parser** that extracts skills, experience, and education, then ranks candidates based on a "Match Score" against a specific JD using Vector Embeddings.

- **Tech Stack:** * **Parsing:** PyResparser or Spacy.
    - **Similarity:** Cosine Similarity, FAISS (Vector Search).
    - **Frontend:** React.js.
- **Expected Output:**
    - A recruiter dashboard that sorts 50+ uploaded PDFs from "Best Match" to "Least Match."
    - Visual "Skill Gap" analysis for each candidate.

Track 9: Decentralized Energy Grid Simulator

Problem Statement: In a smart city, households with solar panels should be able to sell excess electricity to neighbors. Create a **P2P Energy Trading Platform** that uses a bidding algorithm to balance the local grid's supply and demand without a central authority.

- **Tech Stack:** * **Simulation:** Python (Mesa for Agent-based modeling).
    - **Logic:** Game Theory algorithms (Nash Equilibrium).
    - **UI:** D3.js for live grid visualization.
- **Expected Output:**
    - A visual map showing energy flowing from "Producer" houses to "Consumer" houses.
    - A dynamic pricing model that changes based on the time of day and battery storage.

Evaluation Criteria

Teams will be judged on a scale of 100 points based on the following:

Criteria	Weight	Description
Innovation & Logic	25%	How unique and logical is the approach to the problem?
Technical Execution	30%	Code quality, efficient use of tech stack, and model accuracy.
Scalability	15%	Can the system handle 10,000+ users or concurrent data streams?
UX/UI Design	15%	Is the output user-friendly and visually clear?
Presentation	15%	Clarity in explaining the solution and answering jury questions.